

A1:PROPOSAL OF QUESTION

Can we predict the likelihood of a customer churning from the telecommunication industry churn data using the KNN classification method?

A2: DEFINED GOAL

The main goal of the data analysis is to determine whether a customer in a telecommunication industry will churn or not and make recommendations based on the model's results. The K-nearest neighbors (KNN) method will be used to develop a machine learning model to determine which telecommunication industry customers are likely to Churn or not using the key features that will be identified that cause the customer to Churn.

B1:EXPLANATION OF CLASSIFICATION METHOD

The classification method chosen to predict the customers that will Churn is K-nearest neighbors (KNN). According to Christopher, Antony (2021, Feb 2), K-nearest neighbors (KNN) is a supervised learning algorithm for regression and classification. KNN classifies a data point based on how its neighbors are classified.

In the case of the Churn dataset for the telecommunication industry, KNN will analyze the data by comparing each customer's data to the data of other customers in the dataset and make predictions based on the similarity between these points on whether the customer is likely to churn or not.

The KNN method analyzes the selected data set by calculating the distance between data points. The KNN algorithm utilizes distance metrics like Euclidean distance, Manhattan distance, or Minkowski distance to identify the closest data points (neighbors) to a given data point. Once the

distances are calculated, KNN examines the labels of the closest neighbors based on the k parameter (number of neighbors chosen) and predicts and classifies the target data point based on the nearest neighbors.

How the KNN Classification method analyzes the Churn dataset.

KNN analyzes the Churn dataset by calculating the similarity (distance) between data points and assigning a class (churn or no churn) to a target data point based on the majority label of its nearest neighbors. This process ensures that patterns within the dataset drive predictions.

The Expected Outcome from the KNN Model.

The expected output from the KNN analysis is that a model will be created that identifies patterns within the Churn dataset, highlighting key features associated with churn. Based on the features, the customers likely to Churn will be classified.

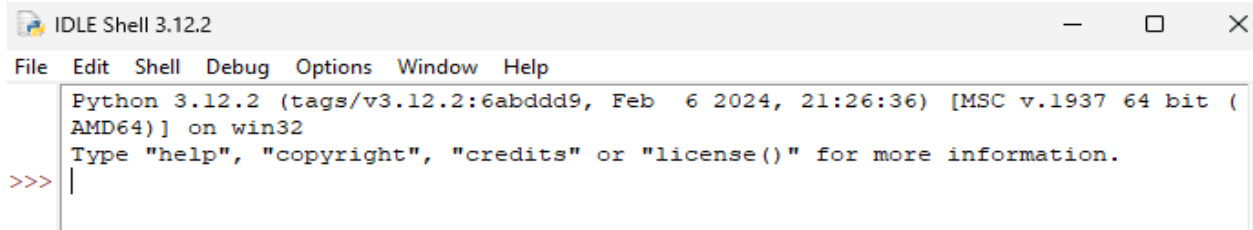
B2: SUMMARY OF METHOD ASSUMPTION

K-Nearest Neighbors (KNN) assumes that the data points close in the feature space are more likely to have similar classification labels or regression values.

B3: PACKAGES OR LIBRARIES LIST

To undertake the analysis, I will be utilizing the Python tool. Python is an open-source general-purpose programming language with many library packages that can be used and shared with others. I utilized Python Programming Language because of its flexibility and extensive library collections, which are valuable for performing complex calculations and analysis.

For Python, I will be using Python version 3.12.2



```
IDLE Shell 3.12.2
File Edit Shell Debug Options Window Help
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb  6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

The following Python libraries will be utilized to perform the various functionalities:

1. Pandas

Pandas will import the data from the CSV file, conduct exploratory data analysis, manipulate data, and convert categorical data to numerical data.

2. Numpy

Numpy will be used for data manipulation.

3. Matplotlib.pyplot

Matplotlib will be used for data visualization.

4. Seaborn

Seaborn will be used for data visualization in the exploratory data analysis, such as creating heatmaps.

5. `from sklearn.model_selection import train_test_split`

The function `train_test_split` from `sklearn.model_selection` will split the dataset into training and testing subsets.

6. `from sklearn import preprocessing`

The library `preprocessing` from `sklearn` will be used for data preparation tasks, such as scaling, encoding categorical variables, and normalizing data.

7. `from sklearn.feature_selection import SelectKBest`

`SelectKBest` from `sklearn.feature_selection` will select the top k features from the dataset based on statistical tests, reducing dimensionality and improving model performance.

8. `from sklearn.feature_selection import f_classif`

The function `f_classif` from `sklearn.feature_selection` will be used to perform the ANOVA F-test to select statistically significant features for classification tasks.

9. `from sklearn.neighbors import KNeighborsClassifier`

The library `KNeighborsClassifier` from `sklearn.neighbors` will be used to implement the K-Nearest Neighbors algorithm for classification tasks.

10. `from sklearn.model_selection import GridSearchCV`

The function `GridSearchCV` from `sklearn.metrics` will perform hyperparameter tuning by testing different combinations of parameters and selecting the best ones for the model.

11. `from sklearn.model_selection import KFold`

The function `KFold` from `sklearn.model_selection` will be used to split the data into `k` consecutive folds for cross-validation, ensuring that the model is trained and validated on different subsets of the data.

12. `from sklearn.metrics import accuracy_score`

The function `accuracy_score` from `sklearn.metrics` will be used to measure the accuracy of a classification model by comparing the predicted labels with the true labels.

13. `from sklearn.metrics import confusion_matrix`

The function `confusion_matrix` from `sklearn.metrics` will create a matrix summarizing the counts of true positives, true negatives, false positives, and false negatives for classification results.

14. `from sklearn.metrics import classification_report`

The function `classification_report` from `sklearn.metrics` will be used to generate a detailed report of classification performance, including precision, recall, F1-score, and support for each class.

15. `from sklearn.metrics import roc_auc_score`

The function `roc_auc_score` from `sklearn.metrics` import will be used to compute the Area Under the Receiver Operating Characteristic Curve (ROC AUC), which measures the model's ability to distinguish between classes.

16. `from sklearn.metrics import roc_curve`

The function `roc_curve` from `sklearn.metrics` will be used to compute the Receiver Operating Characteristic (ROC) curve, which shows the trade-off between sensitivity (recall) and specificity.

17. `from sklearn.metrics import auc`

The function AUC from `sklearn.metrics` will be used to calculate the Area Under the Curve (AUC) for the ROC curve.

18. `from sklearn.metrics import recall_score`

The function `recall_score` from `sklearn.metrics` will measure the proportion of true positives correctly identified.

19. `from sklearn.metrics import f1_score`

The function `f1_score` from `sklearn.metrics` will be used to calculate the F1-score, which balances precision and recall.

20. `from sklearn.metrics import precision_score`

The function `precision_score` from `sklearn.metrics` will measure the proportion of true positives among all predicted positives.

21. warnings.

Additional packages will be used to suppress future warnings and package changes expected from the library.

```
: # 1. Import Libraries.

: !pip install pandas
: !pip install numpy
: !pip install seaborn
: !pip install matplotlib
: !pip install --upgrade seaborn

#D209 Task 1 -KNN Classification Model
#Data Manipulation
import pandas as pd
import numpy as np

#Data visualization
import matplotlib.pyplot as plt
import seaborn as sns

#Machine Learning-KNN model
#Data Preprocessing
from sklearn.model_selection import train_test_split
from sklearn import preprocessing

#Feature Selection
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif

#Modeling
from sklearn.neighbors import KNeighborsClassifier

#Hyperparameter tuning
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import KFold

#Model Evaluation
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.metrics import auc
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import precision_score

#Others
import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore", FutureWarning)
```

C1: DATA PREPROCESSING

After loading the data, it was viewed and reviewed to understand its nature. The following preprocessing was performed on the data.

1. Data type constraint and Data Missingness

Due to the wrong datatype specified while loading the Internet service, the value of Other was identified as missing. To address the issue, the data type was specified as missing data. To address the issue, the value feature was specified as a String.

The data survey variables (Item1, Item2, Item3, Item4, Item5, Item6, Item7, Item8) are Categorical variables loaded as numerical variables. I will convert the data type from numerical to categorical to address the issue.

2. Data Uniqueness

The data was assessed to determine if there were any duplicate values. No duplicate values were found in either column or row.

3. Drop redundant features with high cardinality and features with high collinearity. The features of CaseOrder, Customer_id, Interaction, UID, City, State, County, Zip, Lat, Lng, Job, TimeZone, and Tenure.

4. Cleaning white spaces.

The White spaces were removed through trimming.

5. Renaming the survey columns(Item1, Item2, Item3, Item4, Item5, Item6, Item7, Item8) for a better description.

6. Convert Categorical data to numerical data through dummy encoding (One Hot encoding)

According to Saxena and Shipra (2024, September 18), preprocessing categorical data is necessary since most machine learning models only accept numerical data. One way to do this is through one hot encoding, which is used when the features are nominal, and each category is mapped with a binary variable 0 or 1. 0 represents absence, and 1 represents the presence of that category. The newly created binary features are known as Dummy variables. Dummy encoding uses N-1 features to represent N labels/categories.

7. Standardization and scaling

Since KNN works by measuring the distance between data points, the distance would be dominated by variables with large ranges. To avoid this, the data has been standardized to ensure that all variables are on the same scale and that each variable contributes equally to the distance calculation.

8. Feature selection

SelectKBest will be utilized to select the most important features for this analysis. These are the features with a p_value greater than 0.05. and they contribute the most to predicting the customer's likelihood to churn.

C2: DATA SET VARIABLES

The initial data set variables that will be used to perform the analysis before feature selection include:

1. Churn

The variable Churn is a categorical variable with binary values and will be used as a response (dependent) variable. The purpose of this analysis is to predict the customers who will Churn.

2. Population

The variable Population is numerical. It will be used as a predicting (independent) variable.

3. Area

The variable Area is categorical. It will be used as a predicting (independent) variable.

4. Children

The variable Children is numerical. It will be used as a predicting (independent) variable.

5. Age

The variable Age is numerical. It will be used as a predicting (independent) variable.

6. Income

The variable Income is numerical. It will be used as a predicting (independent) variable.

7. Marital

The variable Marital is categorical. It will be used as a predicting (independent) variable.

8. Gender

The variable Gender is categorical. It will be used as a predicting (independent) variable.

9. Outage_sec_perweek

The variable Outage_sec_perweek is numerical. It will be used as a predicting (independent) variable.

10. Email

The variable Email is numerical. It will be used as a predicting (independent) variable.

11. Contacts

The variable Contacts is numerical. It will be used as a predicting (independent) variable.

12. Yearly_equip_failure

The variable Yearly_equip_failure is numerical. It will be used as a predicting (independent) variable.

13. Techie

The variable Techie is categorical. It will be used as a predicting (independent) variable.

14. Contract

The variable Contract is categorical. It will be used as a predicting (independent) variable.

15. Port_modem

The variable Port_modem is categorical. It will be used as a predicting (independent) variable.

16. Tablet

The variable Tablet is categorical. It will be used as a predicting (independent) variable.

17. InternetService

The variable InternetService is categorical. It will be used as a predicting (independent) variable.

18. Phone

The variable Phone is categorical. It will be used as a predicting (independent) variable.

19. Multiple

The variable Multiple is categorical. It will be used as a predicting (independent) variable.

20. OnlineSecurity

The variable OnlineSecurity is categorical. It will be used as a predicting (independent) variable.

21. OnlineBackup

The variable OnlineBackup is categorical. It will be used as a predicting (independent) variable.

22. DeviceProtection

The variable DeviceProtection is categorical. It will be used as a predicting (independent) variable.

23. TechSupport

The variable TechSupport is categorical. It will be used as a predicting (independent) variable.

24. StreamingTV

The variable StreamingTV is categorical. It will be used as a predicting (independent) variable.

25. StreamingMovies

The variable StreamingMovies is categorical. It will be used as a predicting (independent) variable.

26. PaperlessBilling

The variable PaperlessBilling is categorical. It will be used as a predicting (independent) variable.

27. PaymentMethod

The variable PaymentMethod is categorical. It will be used as a predicting (independent) variable.

28. Tenure

The variable Tenure is numerical. It will be used as a predicting (independent) variable.

29. MonthlyCharge

The variable MonthlyCharge is numerical. It will be used as a predicting (independent) variable.

30. Timelyresponse

The variable Timelyresponse is categorical. It will be used as a predicting (independent) variable.

31. Timelyfixes

The variable Timelyfixes is categorical. It will be used as a predicting (independent) variable.

32. Timelyreplacements

The variable Timelyreplacements is categorical. It will be used as a predicting (independent) variable.

33. Reliability

The variable Reliability is categorical. It will be used as a predicting (independent) variable.

34. Options

The variable Options is categorical. It will be used as a predicting (independent) variable.

35. Respectfulresponse

The variable Respectfulresponse is categorical. It will be used as a predicting (independent) variable.

36. Courteousexchange

The variable Courteousexchange is categorical. It will be used as a predicting (independent) variable.

37. Evidenceofactivelistening

The variable Evidenceofactivelistening is categorical. It will be used as a predicting (independent) variable.

C3: STEPS FOR ANALYSIS

In performing the analysis, the following steps were undertaken

1. Loading the necessary libraries and packages that will be utilized for the analysis

```
: # 1. Import Libraries.

: !pip install pandas
: !pip install numpy
: !pip install seaborn
: !pip install matplotlib
: !pip install --upgrade seaborn

#D209 Task 1 -KNN Classification Model
#Data Manipulation
import pandas as pd
import numpy as np

#Data visualization
import matplotlib.pyplot as plt
import seaborn as sns

#Machine Learning-KNN model
#Data Preprocessing
from sklearn.model_selection import train_test_split
from sklearn import preprocessing

#Feature Selection
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif

#Modeling
from sklearn.neighbors import KNeighborsClassifier

#Hyperparameter tuning
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import KFold

#Model Evaluation
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.metrics import auc
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import precision_score

#Others
import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore", FutureWarning)
```

2. Loading the data into a dataframe

```
] : #File path variables

Local_path = 'C:/Users/beatr/OneDrive/Desktop/D209/Task1/'

InputFilePath = Local_path + 'churn_clean.csv'

OutputfilePath = Local_path + 'churn_cleaned_df.csv'

XtrainOutputfilePath =Local_path + 'Xtrain.csv'

XtestOutputfilePath =Local_path + 'Xtest.csv'

ytrainOutputfilePath =Local_path + 'ytrain.csv'

ytestOutputfilePath =Local_path + 'ytest.csv'

] : # 2. Load the data.

] : #Loading the data
#Churn_df = pd.read_csv(InputFilePath,index_col=0)
Churn_df = pd.read_csv(InputFilePath)
```

3. Viewing the data

D209: Data Mining 1_KNN classification
BeatriceM_WGU_MSDA 16

```
#Initial data exploration (Understanding the data)
```

```
pd.set_option('display.max_columns', None)
Churn_df.head()
```

	CaseOrder	Customer_id	Interaction	UID	City	State	County	Zip	
0	1	K409198	aa90260b-4141-4a24-8e36-b04ce1f4f77b	e885b299883d4f9fb18e39c75155d990	Point Baker	AK	Prince of Wales-Hyder	99927	56.25
1	2	S120509	fb76459f-c047-4a9d-8af9-e0f7d4ac2524	f2de8bef964785f41a2959829830fb8a	West Branch	MI	Ogemaw	48661	44.32
2	3	K191035	344d114c-3736-4be5-98f7-c72c281e2d35	f1784cfa9f6d92ae816197eb175d3c71	Yamhill	OR	Yamhill	97148	45.35
3	4	D90850	abfa2b40-2d43-4994-b15a-989b8c79e311	dc8a365077241bb5cd5ccd305136b05e	Del Mar	CA	San Diego	92014	32.96
4	5	K662701	68a861fd-0d20-4e51-a587-8a90407ee574	aabb64a116e83fdc4befc1fbab1663f9	Needville	TX	Fort Bend	77461	29.38

```
Churn_df.shape
```

```
(10000, 50)
```

4. Evaluating the data and data structure


```
Churn_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CaseOrder                            10000 non-null  int64
1   Customer_id                          10000 non-null  object
2   Interaction                           10000 non-null  object
3   UID                                  10000 non-null  object
4   City                                 10000 non-null  object
5   State                                10000 non-null  object
6   County                               10000 non-null  object
7   Zip                                  10000 non-null  int64
8   Lat                                  10000 non-null  float64
9   Lng                                  10000 non-null  float64
10  Population                           10000 non-null  int64
11  Area                                 10000 non-null  object
12  TimeZone                             10000 non-null  object
13  Job                                   10000 non-null  object
14  Children                             10000 non-null  int64
15  Age                                  10000 non-null  int64
16  Income                               10000 non-null  float64
17  Marital                              10000 non-null  object
18  Gender                               10000 non-null  object
19  Churn                                10000 non-null  object
20  Outage_sec_perweek                   10000 non-null  float64
21  Email                                10000 non-null  int64
22  Contacts                             10000 non-null  int64
23  Yearly equip_failure                 10000 non-null  int64
24  Techie                               10000 non-null  object
25  Contract                             10000 non-null  object
26  Port_modem                           10000 non-null  object
27  Tablet                               10000 non-null  object
28  InternetService                      7871 non-null   object
29  Phone                                10000 non-null  object
30  Multiple                             10000 non-null  object
31  OnlineSecurity                       10000 non-null  object
32  OnlineBackup                         10000 non-null  object
33  DeviceProtection                     10000 non-null  object
34  TechSupport                          10000 non-null  object
35  StreamingTV                          10000 non-null  object
36  StreamingMovies                      10000 non-null  object
37  PaperlessBilling                     10000 non-null  object
38  PaymentMethod                        10000 non-null  object
39  Tenure                               10000 non-null  float64
40  MonthlyCharge                        10000 non-null  float64
41  Bandwidth_GB_Year                   10000 non-null  float64
42  Item1                                10000 non-null  int64
43  Item2                                10000 non-null  int64
44  Item3                                10000 non-null  int64
45  Item4                                10000 non-null  int64
46  Item5                                10000 non-null  int64
47  Item6                                10000 non-null  int64
48  Item7                                10000 non-null  int64
49  Item8                                10000 non-null  int64
dtypes: float64(7), int64(16), object(27)
memory usage: 3.8+ MB
```

5. Data Cleaning and preprocessing

This includes

- i. Checking for Nulls and Missing data.

```
#Checking for Nulls and missing data
#print(Churn_df.isna().sum().sort_values())
print(Churn_df[Churn_df[Churn_df.columns[Churn_df.isna().any()]].columns].info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   InternetService  7871 non-null   object
dtypes: object(1)
memory usage: 78.3+ KB
None
```

```
#Cleaning the data
# Replace NaN with "Other" in the 'InternetServices' column
Churn_df['InternetService'] = Churn_df['InternetService'].fillna('Other')
```

- ii. Cleaning white spaces

```
#Trim and Clean White Spaces
Churn_df = Churn_df.map(lambda x: x.strip() if isinstance(x, str) else x)
```

- iii. Renaming columns for better descriptions

```
# Rename Last 8 survey columns for better description of variables
Churn_df.rename(columns = {'Item1':'Timelyresponse',
                           'Item2':'Timelyfixes',
                           'Item3':'Timelyreplacements',
                           'Item4':'Reliability',
                           'Item5':'Options',
                           'Item6':'Respectfulresponse',
                           'Item7':'Courteousexchange',
                           'Item8':'Evidenceofactivelistening'},
                inplace=True)
```

- iv. Checking for duplicates

```
: #Checking for duplicated data

print("Duplicated rows")
print(Churn_df.duplicated().any())
print(" ")
print("Duplicated columns")
print(Churn_df.columns.duplicated().any())

Duplicated rows
False

Duplicated columns
False
```

- v. Checking the number of unique values per feature (Checking for high cardinality)

```
#Checking for unique values
```

```
Churn_df.nunique()
```

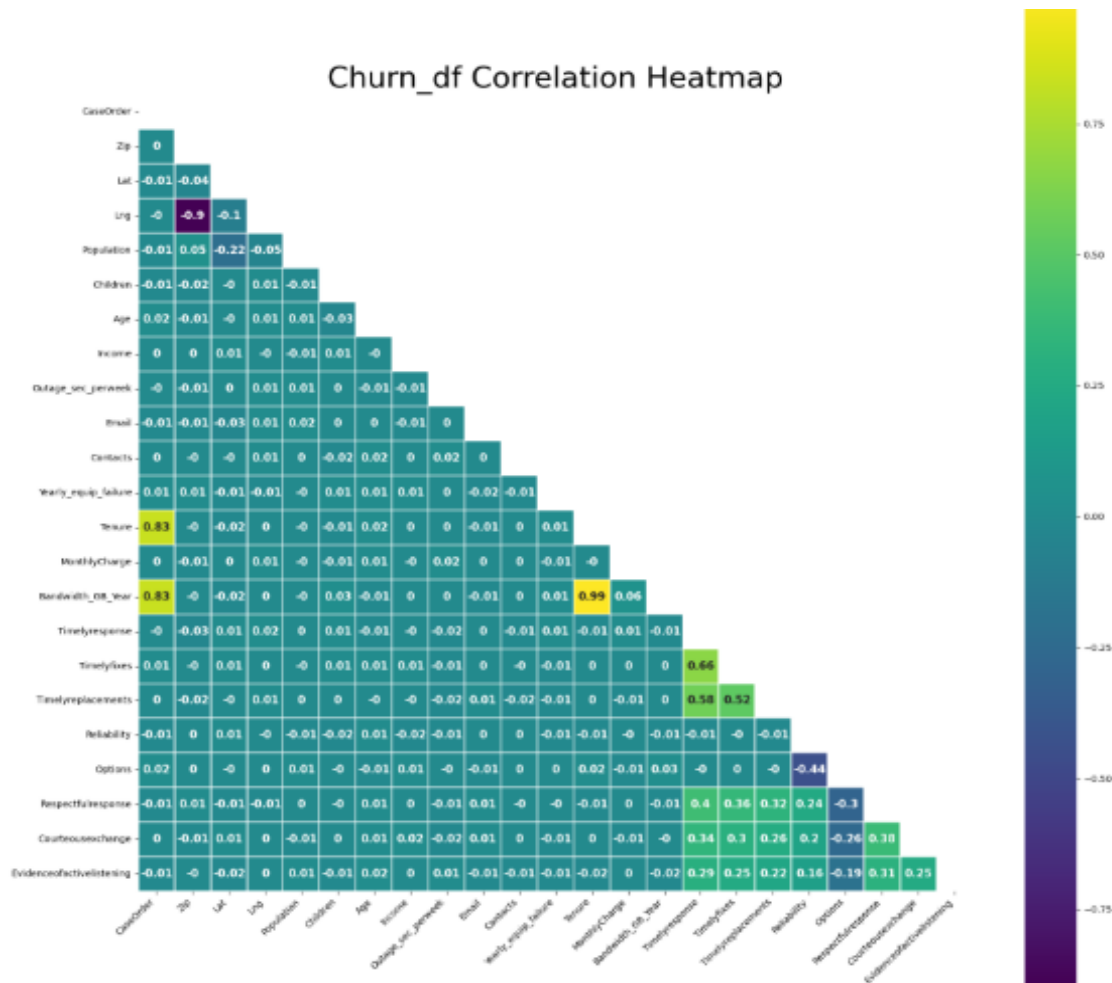
CaseOrder	10000
Customer_id	10000
Interaction	10000
UID	10000
City	6058
State	52
County	1620
Zip	8583
Lat	8563
Lng	8630
Population	5933
Area	3
TimeZone	25
Job	639
Children	11
Age	72
Income	9993
Marital	5
Gender	3
Churn	2
Outage_sec_perweek	9986
Email	23
Contacts	8
Yearly_equip_failure	6
Techie	2
Contract	3
Port_modem	2
Tablet	2
InternetService	3
Phone	2
Multiple	2
OnlineSecurity	2
OnlineBackup	2
DeviceProtection	2
TechSupport	2
StreamingTV	2
StreamingMovies	2
PaperlessBilling	2
PaymentMethod	4
Tenure	9996
MonthlyCharge	750
Bandwidth_GB_Year	10000
Timelyresponse	7
Timelyfixes	7
Timelyreplacements	8
Reliability	7
Options	7
Respectfulresponse	8
Courteousexchange	7
Evidenceofactivelistening	8

dtype: int64

vi. Checking for correlation

```
#Analysing the Correlation /Relationship analysis
Continous_columns = Churn_df.select_dtypes(include=['float64', 'int64']).columns.tolist()
Cor_data = Churn_df[Continous_columns]

def Generate_heatmap_graph(corr, chart_title, mask_uppertri=False):
    """ Based on features, generate a correlation matrix """
    mask = np.zeros_like(corr, dtype=bool)
    mask[np.triu_indices_from(mask)] = mask_uppertri
    fig, ax = plt.subplots(figsize=(20, 20))
    sns.heatmap(corr,
                mask=mask,
                square=True,
                annot=True,
                annot_kws={'size': 12, 'rotation': 0, 'weight': 'bold'},
                cmap=plt.get_cmap('viridis'),
                linewidths=.1,
                ax=ax)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right') # Rotate x-axis labels
    plt.title(chart_title, fontsize=32)
    plt.show()
var_corr = round(Cor_data.corr(), 2)
Generate_heatmap_graph(var_corr, chart_title='Churn_df Correlation Heatmap', mask_uppertri=True)
```



- vii. Dropping features that will not be used for analysis due to either redundancy, high cardinality, or high collinearity.

#Dropping Columns that will not be used in the analysis

```
Churn df = Churn df.drop (columns = ['CaseOrder', 'Customer id', 'Interaction', 'UID', 'City', 'State', 'County', 'Zip', 'Lat', 'Lng', 'Job', 'TimeZone', 'Bandwidth GB Year'])
```

- viii. Dummy encoding of categorical variables

Converting Categorical data to numerical data through dummy encoding. Include all dummies for non-binary variables and include only one dummy variable for binary columns.

```
#Converting Categorical data to numerical data through dummy encoding (One Hot encoding)

categorical_columns = Churn_df.select_dtypes(include=['object']).columns.tolist()

# Separate binary and non-binary categorical columns
binary_columns = [col for col in categorical_columns if Churn_df[col].nunique() == 2]
non_binary_columns = [col for col in categorical_columns if Churn_df[col].nunique() > 2]

# Create dummy variables
Churn_df_encoded = pd.get_dummies(
    Churn_df,
    columns=non_binary_columns, # Include all dummies for non-binary variables
    dummy_na=False,
    drop_first=False, # Do not drop any dummy for non-binary
    dtype='int64'
)

# For binary columns, include only one dummy variable
Churn_df_encoded = pd.get_dummies(
    Churn_df_encoded,
    columns=binary_columns,
    dummy_na=False,
    drop_first=True, # Drop the first dummy for binary
    dtype='int64'
)

# Display the resulting dataframe
Churn_df_encoded.head()
```

ix. Standardization and scaling of the data

Standardization and scaling will be used to transform data to improve model performance by ensuring features have similar ranges. Using Python's preprocessing library, MinMaxScaler scales data to a specified range (default: 0 to 1) by subtracting the minimum value and dividing by the range, preserving the original data distribution.

```
# 5. Standardization (includes scaling, normalization, data types, etc., as applicable)

# Separate the X variables (explanatory) from the Y variable (response) into different dataframes
df_X = Churn_df_encoded.drop(["Churn_Yes"], axis=1).copy()
df_y = Churn_df_encoded["Churn_Yes"].copy()

# Standardize the X variables (explanatory) to put all on the same scale for KNN to measure distance
df_X = pd.DataFrame(preprocessing.MinMaxScaler().fit_transform(df_X), columns=df_X.columns)
df_X.head(20)
```

x. Feature selection.

Feature selection using SelectKBest involves selecting the top k features based on a statistical test to improve model performance and reduce overfitting. In this case, features with a $p_value < 0.05$ will be selected.

```
# 6. Feature Selection - select the variables you need for your analysis.

# Use SelectKBest to determine the best features to include in the model from the 45 features we're starting with
#Selecting features with a p_value <0.05. Indicate strong evidence against the null hypothesis,
#suggesting that the observed result is unlikely to occur by chance.
feat_select = SelectKBest(f_classif, k='all')
feat_select.fit_transform(df_X, df_y)
feat_pvals = pd.DataFrame({'Feature' : df_X.columns, 'p_value' : feat_select.pvalues_}).sort_values('p_value')
Selected_Features=feat_pvals[feat_pvals['p_value'] < 0.05]

#print(Selected_Features[['Feature', 'p_value']])

Selected_Features = Selected_Features.copy()
Selected_Features['p_value_rounded'] = Selected_Features['p_value'].round(10)
pd.set_option('display.float_format', '{:.10f}'.format)
pd.set_option('display.max_columns', None)
print(Selected_Features[['Feature', 'p_value_rounded']])
pd.reset_option('display.float_format')
```

	Feature	p_value_rounded
8	Tenure	0.0000000000
9	MonthlyCharge	0.0000000000
32	StreamingMovies_Yes	0.0000000000
31	StreamingTV_Yes	0.0000000000
20	Contract_Two Year	0.0000000000
19	Contract_One year	0.0000000000
26	Multiple_Yes	0.0000000000
18	Techie_Yes	0.0000000000
23	InternetService_Fiber Optic	0.0000000049
29	DeviceProtection_Yes	0.0000000158
28	OnlineBackup_Yes	0.0000004339
24	InternetService_Other	0.0001599912
35	PaymentMethod_Electronic Check	0.0027744607
16	Gender_Male	0.0050114023
25	Phone_Yes	0.0085439734
77	Courteousexchange_4	0.0326455463

6. Exporting the cleaned data

```
# 7. Export the Cleaned Data
```

```
columns = Selected_Features['Feature'].values.tolist()
# Recombine df_X (feature set) and df_y (response variable) to save the whole thing in one dataframe
Churn_Cleaned_df = df_X[columns].copy()
#Churn_Cleaned_df.insert(22, column="Churn_Yes", value=df_y.reset_index(drop=True))
# Save dataframe to CSV, ignore index (if included, this will create an additional unnecessary column)
Churn_Cleaned_df['Churn_Yes'] = df_y.reset_index(drop=True)
Churn_Cleaned_df.to_csv(OutputfilePath, index=False)
```

```
print('The Cleaned Data')
print(f'Churn_Cleaned_df.shape: {Churn_Cleaned_df.shape}')
Churn_Cleaned_df.info()
```

```
The Cleaned Data
Churn_Cleaned_df.shape: (10000, 19)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 19 columns):
 #   Column                                     Non-Null Count  Dtype
---  -
 0   Tenure                                   10000 non-null  float64
 1   MonthlyCharge                           10000 non-null  float64
 2   StreamingMovies_Yes                     10000 non-null  float64
 3   Contract_Month-to-month                 10000 non-null  float64
 4   StreamingTV_Yes                         10000 non-null  float64
 5   Contract_Two Year                       10000 non-null  float64
 6   Contract_One year                       10000 non-null  float64
 7   Multiple_Yes                            10000 non-null  float64
 8   InternetService_DSL                     10000 non-null  float64
 9   Techie_Yes                             10000 non-null  float64
10   InternetService_Fiber Optic             10000 non-null  float64
11   DeviceProtection_Yes                    10000 non-null  float64
12   OnlineBackup_Yes                        10000 non-null  float64
13   InternetService_Other                   10000 non-null  float64
14   PaymentMethod_Electronic Check          10000 non-null  float64
15   Gender_Female                           10000 non-null  float64
16   Phone_Yes                               10000 non-null  float64
17   Courteousexchange_4                     10000 non-null  float64
18   Churn_Yes                               10000 non-null  int64
dtypes: float64(18), int64(1)
memory usage: 1.4 MB
```

C4: CLEANED DATA SET

A copy of the cleaned data set.



churn_cleaned_df.csv

D1:SPLITTING THE DATA

Split the data into training and test data sets using `train_test_split`. The data will be split into `X_train`, `X_test`, `y_train`, and `y_test` to separate training data that will be used to build the model from testing data that will be used to evaluate the model's performance. This will ensure the model is trained and validated on distinct datasets, preventing overfitting and generalizing well to new data. The data will be split into training and testing subsets, with 80% allocated for training and 20% for testing; stratification will ensure that class proportions are maintained across both subsets.

```
# 8. Split the Data - split the cleaned data.

# Set X and y (standard sklearn variables) to equal the selected X features and the y response of interest
X = Churn_Cleaned_df.drop(["Churn_Yes"], axis=1).copy()
y = Churn_Cleaned_df["Churn_Yes"].copy()

print('Before Data Split')
print(f'X.shape: {X.shape}')
print(f'y.shape: {y.shape}')

Before Data Split
X.shape: (10000, 18)
y.shape: (10000,)

# Split the data into train and test sets, 80% train, 20% test, use stratify to maintain proportions across split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.8, test_size=0.2, random_state = 42, stratify = y)

print("After data Split")
print(f'X_train.shape: {X_train.shape}')
print(f'X_test.shape: {X_test.shape}')
print(f'y_train.shape: {y_train.shape}')
print(f'y_test.shape: {y_test.shape}')

After data Split
X_train.shape: (8000, 18)
X_test.shape: (2000, 18)
y_train.shape: (8000,)
y_test.shape: (2000,)

# Save each of the split data for Xtrain,Xtest,ytrain,y,test
X_train.to_csv(XtrainOutputfilePath, index=False)
X_test.to_csv(XtestOutputfilePath, index=False)
y_train.to_csv(ytrainOutputfilePath, index=False)
y_test.to_csv(ytestOutputfilePath, index=False)
```

The file output for the split data



Xtrain.csv



Xtest.csv



ytrain.csv



ytest.csv

D2:OUTPUT AND INTERMEDIATE CALCULATIONS

The KNN machine learning algorithm was used to perform the classification

1. Initializing the model

The model was initialized using the KNeighborsClassifier

```
knn = KNeighborsClassifier()
```

2. Train and fit the model

After initializing the model, the KNN classifier was trained using the fit method with the training data X_train (features) and y_train (labels or target variable). The fit function lets the model learn patterns and relationships from the training data.

```
knn.fit(X_train, y_train)
```

```
▼ KNeighborsClassifier ① ②  
KNeighborsClassifier()
```

3. Evaluating the model and the Output

After training the model, it was evaluated for its performance on the testing data(unseen data) using metrics such as accuracy, precision, recall, or other relevant evaluation criteria. The predictions and probabilities were generated

```
# 11. Evaluate the model for Accuracy and check the error metrics (includes accuracy,
#precision, recall, F1-score, ROC-AUC)

#predictions
y_pred = knn.predict(X_test)
y_proba = knn.predict_proba(X_test)[:,1]

# Evaluate model
#Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Precision, Recall, F1-Score
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

best_params = {'n_neighbors': knn.n_neighbors, 'weights': knn.weights}
best_score = f1

print("Best Parameters:", best_params)
print("Best F1-Score:", round(best_score, 2))
print(" ")
# ROC-AUC
roc_auc = roc_auc_score(y_test, y_proba)
print("\033[1m\033[4m\033[38;2;0;0;0mInitial Model Evaluation Metrics:\033[0m")

print(f'Initial Accuracy: {round(accuracy,2)}')
print(f'Initial Precision: {round(precision,2)}')
print(f'Initial Recall: {round(recall,2)}')
print(f'Initial F1 Score: {round(f1,2)}')
print(f'Initial ROC-AUC: {roc_auc:.2f}')
print(" ")
# Print training and testing accuracy
print("\033[1m\033[4m\033[38;2;0;0;0mInitial Model Accuracy Scores:\033[0m")
print(f"The training accuracy of this KNN classification Initial model is {knn.score(X_train, y_train)}")
print(f"The testing accuracy of this KNN classification Initial model is {knn.score(X_test, y_test)}")

print(" ")
print("\033[1m\033[4m\033[38;2;0;0;0mKNN Initial Model Classification Report:\033[0m")
print("Classification_Report:\n", classification_report(y_test, y_pred))

#print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
initialMatrix=confusion_matrix(y_test, y_pred)
print(" ")
# Plot confusion matrix using seaborn heatmap
print("\033[1m\033[4m\033[38;2;0;0;0mInitial Confusion Matrix:\033[0m")
plt.figure(figsize=(6, 5))
sns.heatmap(initialMatrix, annot=True, fmt="d", cmap="plasma", xticklabels=["Predicted No Churn", "Predicted Churn"], yticklabels=["Actual No Churn", "Actual Churn"])
plt.title(" The Confusion Matrix -The initial KNN Model")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print(" ")
# Plot the ROC curve
print("\033[1m\033[4m\033[38;2;0;0;0mInitial ROC Curve:\033[0m")
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
plt.plot([0, 1], [0, 1], 'k--')
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for KNN Classification')
plt.legend(loc="lower right")
plt.show()
```

The initial model utilized 5 neighbors and uniform distance as the best parameters.

Neighbors are the data points closest to the point the model is trying to predict or classify.

Uniform distance is a distance weighing method where each neighbor contributes equally to the prediction.

The output

F1-score is a metric used to measure the balance between precision and recall in a classification model. The best F1 score achieved was 0.69. This indicates a good balance between precision and recall.

The model's training accuracy was 0.89, and testing accuracy was 0.85. This means that the model is performing well but slightly overfitting the data as it performs better on the training data than on the testing data, which is unseen data. The accuracy was 0.85, meaning the model correctly predicted 85% of the instances (positive and negative cases).

The model achieved a precision of 0.77, meaning it predicted positive instances correctly 77% of the time.

The model achieved a recall or sensitivity or true positive rate of 0.63, meaning that the model correctly identified 63% of all positive cases.

The model achieved an AUC of 0.88, meaning that the model does a good job distinguishing between the classes.

When the confusion Matrix was generated, the model correctly predicted 1370 negative instances(True Negatives), and the model incorrectly predicted 100 instances as positive (False positives). The model incorrectly predicted 195 instances as negative (False negatives). The model correctly predicted 335 positive instances (True Positives)

Best Parameters: {'n_neighbors': 5, 'weights': 'uniform'}
Best F1-Score: 0.69

Initial Model Evaluation Metrics:

Initial Accuracy: 0.85
Initial Precision: 0.77
Initial Recall: 0.63
Initial F1 Score: 0.69
Initial ROC-AUC: 0.88

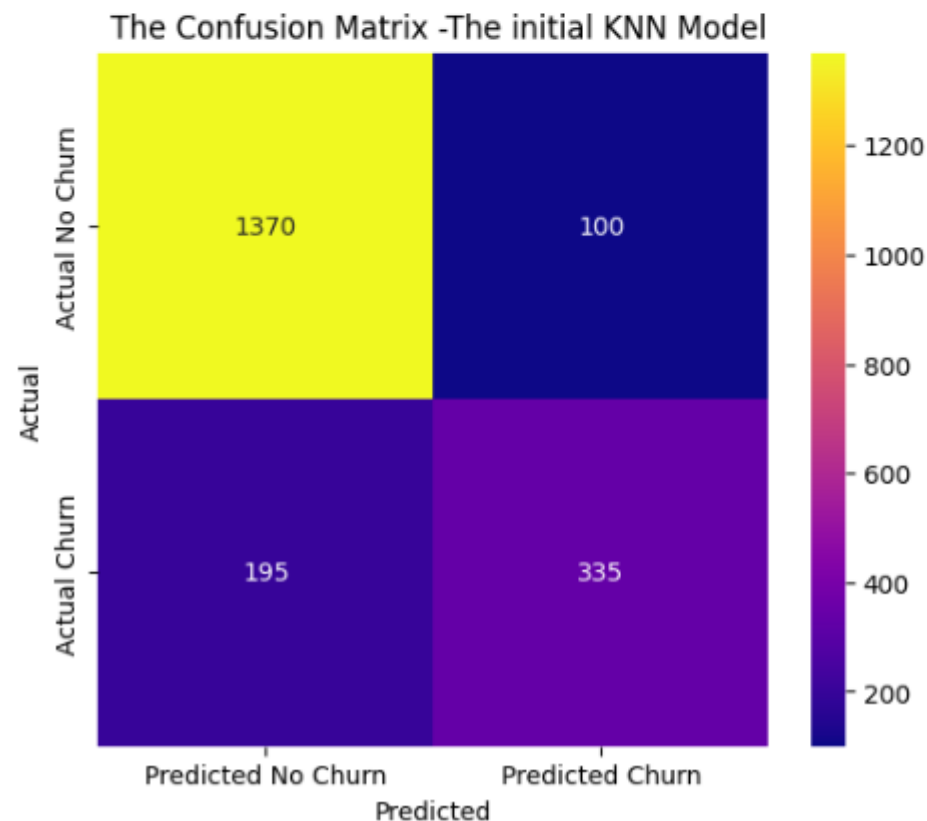
Initial Model Accuracy Scores:

The training accuracy of this KNN classification Initial model is 0.8945
The testing accuracy of this KNN classification Initial model is 0.8525

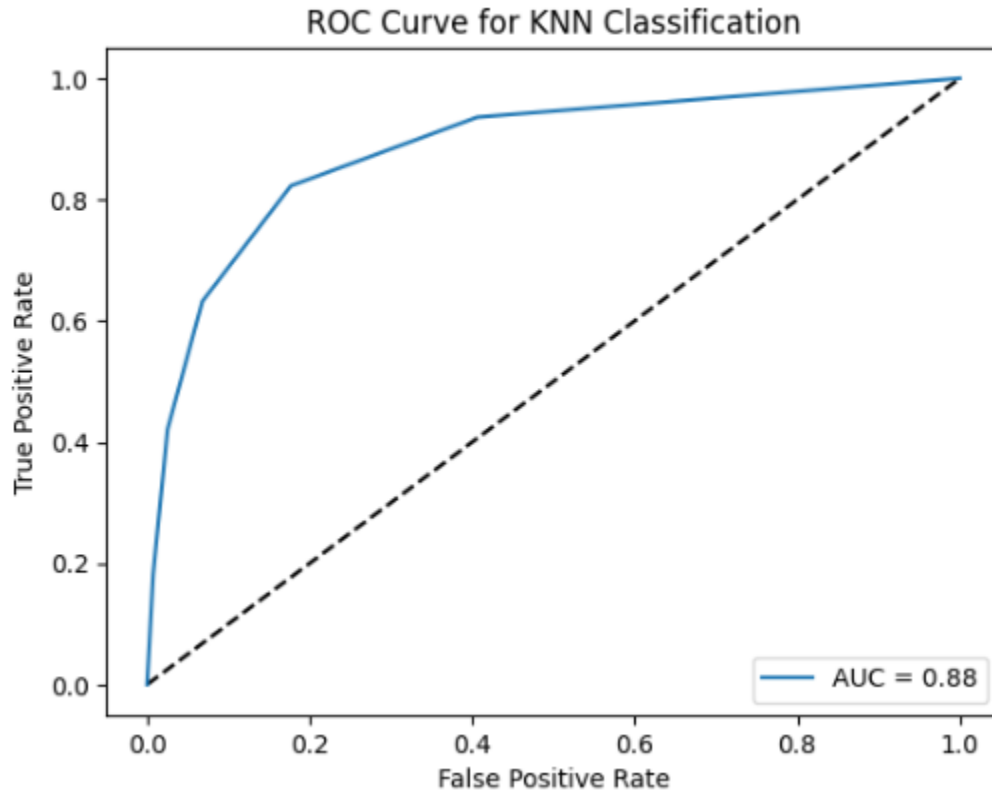
KNN Initial Model Classification Report:

Classification_ Report:	precision	recall	f1-score	support
0	0.88	0.93	0.90	1470
1	0.77	0.63	0.69	530
accuracy			0.85	2000
macro avg	0.82	0.78	0.80	2000
weighted avg	0.85	0.85	0.85	2000

Initial Confusion Matrix:



Initial ROC Curve:



4. Optimizing the model

GridSearchCV has been used to optimize the model to find the best hyperparameters. It is a technique used for hyperparameter tuning in machine learning models. It exhaustively searches through a specified parameter grid (set of possible values) to find the best combination of hyperparameters for a given model.

In optimizing the model, the parameter grid included values of k ranging from 1 to 50, aiming to identify the optimal number of neighbors for KNN. The GridSearchCV was used with 5-fold cross-validation. The grid search evaluated 49 candidates using 5-fold cross-validation, totaling 245 model fits and F1-score as the evaluation metric to select the best-performing model. The best parameters were identified as $n_neighbors = 11$, achieving a top F1-score of 0.68.

```
# 12. Optimize the model (includes hyperparameter tuning or cross-validation).

# Determine what is the best number of neighbors to use for KNN classification, from values of k = 1 to k = 50.
# Determine the weighing strategy of the neighbor whether its is uniform where all neighbors contribute equally or
# it is distance where neighbors are weighed based on the distance and search for the best metric to calculate the distance.

param_grid = {
    "n_neighbors": np.arange(1, 50),
    # "weights": ["uniform", "distance"],
    # "metric": ["euclidean", "manhattan"]
}

# Initialize GridSearchCV
grid_search = GridSearchCV(
    estimator=KNeighborsClassifier(),
    param_grid=param_grid,
    cv=5, # fold cross-validation
    scoring="f1", # Use F1-Score for evaluation
    verbose=1
)

# Perform grid search
grid_search.fit(X_train, y_train)

# Best parameters and score
best_params = grid_search.best_params_
best_score = grid_search.best_score_

print("Best Parameters:", best_params)
print("Best F1-Score:", round(best_score,2))

# Use the best model
best_knn = grid_search.best_estimator_

Fitting 5 folds for each of 49 candidates, totalling 245 fits
Best Parameters: {'n_neighbors': 11}
Best F1-Score: 0.68
```

The output of the final model

F1-score is a metric used to measure the balance between precision and recall in a classification mode. The final F1-score achieved was 0.71. This indicates a good balance between precision and recall.

The model's training accuracy was 0.87, and the testing accuracy was 0.86. This means that the model performs well on the training and testing data, which is unseen data. This is an improvement from the initial model, which had overfitting.

The accuracy was 0.86, meaning the model correctly predicted 86% of the instances (positive and negative cases).

The model achieved a precision of 0.81, meaning that it predicted positive instances correctly 81% of the time.

The model achieved a recall or sensitivity or true positive rate of 0.63, meaning that the model correctly identified 63% of all positive cases.

The model achieved an AUC of 0.91, meaning that the model does a good job distinguishing between the classes.

When the confusion Matrix was generated, the model correctly predicted 1390 negative instances(True Negatives), and the model incorrectly predicted 80 instances as positive (False positives). The model incorrectly predicted 194 instances as negative (False negatives). The model correctly predicted 336 positive instances (True Positives).


```
# Forecast or Predict using the optimized KNN model
final_predictions = best_knn.predict(X_test)
final_proba = best_knn.predict_proba(X_test)[: , 1]

# Evaluate the final model
final_accuracy = accuracy_score(y_test, final_predictions)
final_precision = precision_score(y_test, final_predictions)
final_recall = recall_score(y_test, final_predictions)
final_f1 = f1_score(y_test, final_predictions)
final_roc_auc = roc_auc_score(y_test, final_proba)

# Print final metrics
print("\033[1m\033[4m\033[38;2;0;0;0mFinal Model Evaluation Metrics:\033[0m")
print(f"Final Accuracy: {final_accuracy:.2f}")
print(f"Final Precision: {final_precision:.2f}")
print(f"Final Recall: {final_recall:.2f}")
print(f"Final F1-Score: {final_f1:.2f}")
print(f"Final ROC-AUC: {final_roc_auc:.2f}")

# Print training and testing accuracy
print("\033[1m\033[4m\033[38;2;0;0;0mFinal Model Accuracy Scores:\033[0m")
print(f"The training accuracy of this KNN classification Final model is {best_knn.score(X_train, y_train)}")
print(f"The testing accuracy of this KNN classification Final model is {best_knn.score(X_test, y_test)}")

# Print the classification report
print("\033[1m\033[4m\033[38;2;0;0;0mKNN final Model Classification Report:\033[0m")
print(classification_report(y_test, final_predictions))

# Generate the confusion matrix
final_matrix = confusion_matrix(y_test, final_predictions)

# Plot confusion matrix using seaborn heatmap
print("\033[1m\033[4m\033[38;2;0;0;0mFinal Confusion Matrix:\033[0m")
plt.figure(figsize=(6, 5))
sns.heatmap(final_matrix, annot=True, fmt="d", cmap="plasma",
            xticklabels=["Predicted No Churn", "Predicted Churn"],
            yticklabels=["Actual No Churn", "Actual Churn"])
plt.title("Confusion Matrix - KNN Model")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Plot the ROC curve
print("\033[1m\033[4m\033[38;2;0;0;0mFinal ROC Curve:\033[0m")
fpr, tpr, thresholds = roc_curve(y_test, final_proba)
plt.plot([0, 1], [0, 1], 'k--')
plt.plot(fpr, tpr, label=f"AUC = {final_roc_auc:.2f}")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for KNN Classification')
plt.legend(loc="lower right")
plt.show()
```

Final Model Evaluation Metrics:

Final Accuracy: 0.86
Final Precision: 0.81
Final Recall: 0.63
Final F1-Score: 0.71
Final ROC-AUC: 0.91

Final Model Accuracy Scores:

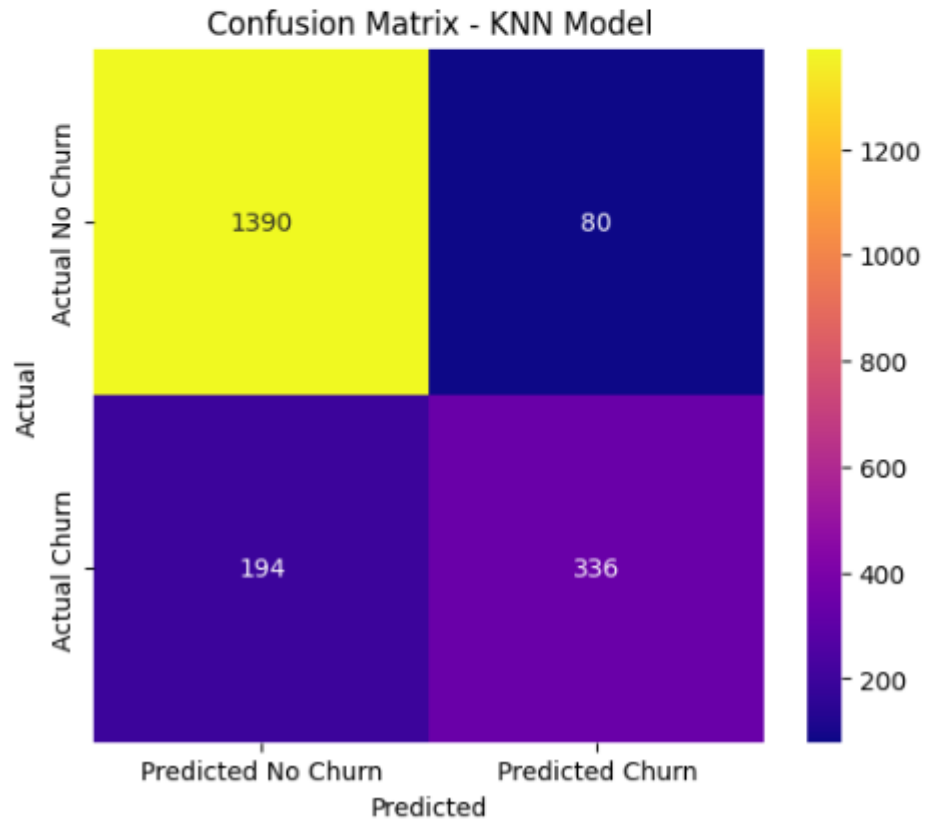
The training accuracy of this KNN classification Final model is 0.8785

The testing accuracy of this KNN classification Final model is 0.863

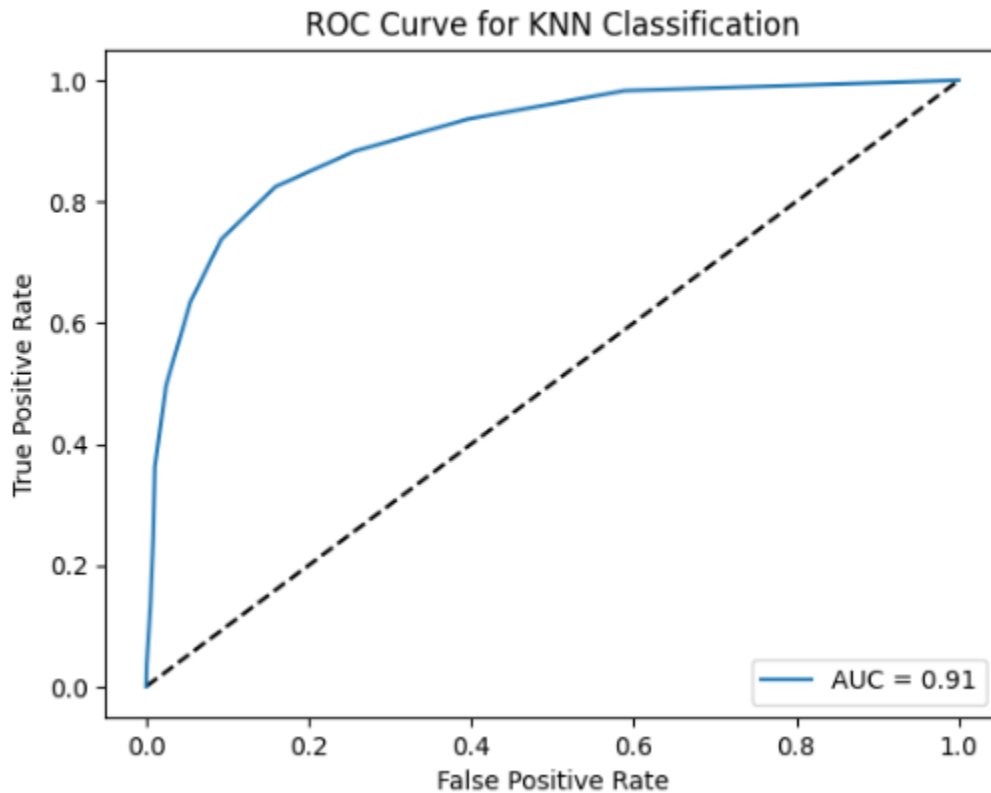
KNN final Model Classification Report:

	precision	recall	f1-score	support
0	0.88	0.95	0.91	1470
1	0.81	0.63	0.71	530
accuracy			0.86	2000
macro avg	0.84	0.79	0.81	2000
weighted avg	0.86	0.86	0.86	2000

Final Confusion Matrix:



Final ROC Curve:



D3:CODE EXECUTION

Please modify the local file path

Local_path = 'C:/Users/beatr/OneDrive/Desktop/D209/Task1/'

```
#File path variables
Local_path = 'C:/Users/beatr/OneDrive/Desktop/D209/Task1/'

InputFilePath = Local_path + 'churn_clean.csv'
OutputfilePath = Local_path + 'churn_cleaned_df.csv'
XtrainOutputfilePath = Local_path + 'Xtrain.csv'
XtestOutputfilePath = Local_path + 'Xtest.csv'
ytrainOutputfilePath = Local_path + 'ytrain.csv'
ytestOutputfilePath = Local_path + 'ytest.csv'
```



BeatriceMwangi_D2
09_DataMining1-Ta:

E1: ACCURACY AND AUC

Accuracy

Accuracy is a metric used to evaluate the performance of a classification model. It measures the proportion of correctly classified instances from the total number of instances in the data set. According to Dilmegani, Cem(2024, October 28), accuracy is a metric in machine learning that measures the overall correctness of a model's prediction.

Initial Model Accuracy Scores:

The training accuracy of this KNN classification Initial model is 0.8945

The testing accuracy of this KNN classification Initial model is 0.8525

Final Model Accuracy Scores:

The training accuracy of this KNN classification Final model is 0.8785

The testing accuracy of this KNN classification Final model is 0.863

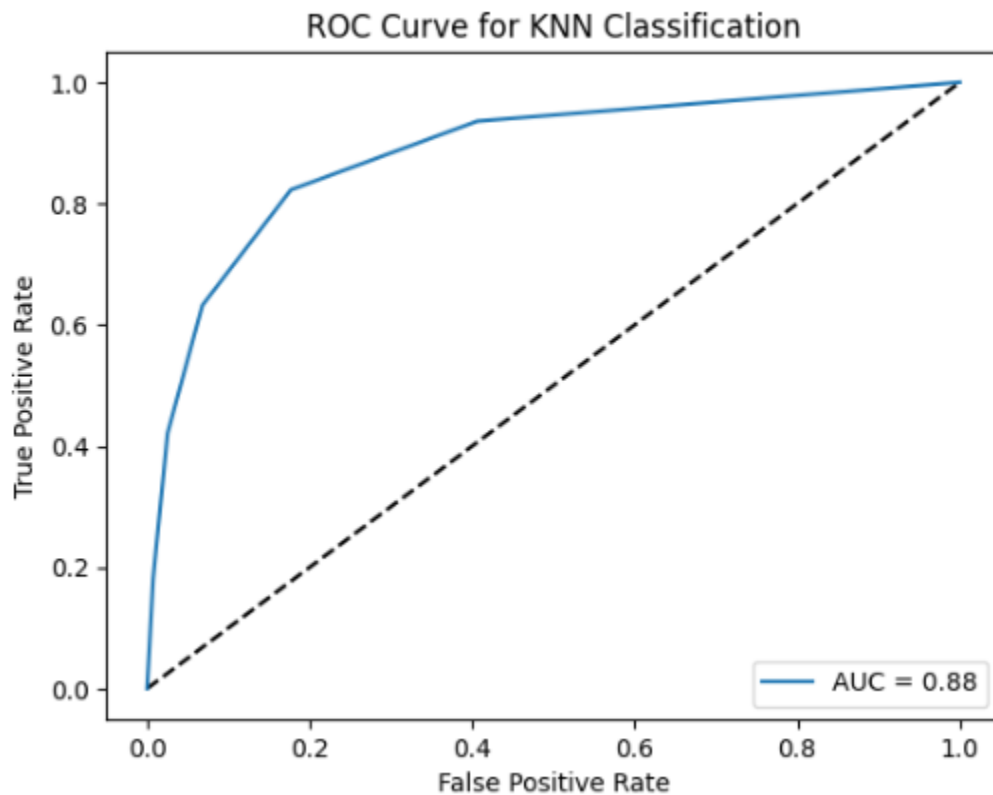
In the initial model accuracy, the model correctly classified 89% of the training sample data. The high score indicates that the model fits the training data very well. However, the testing accuracy is lower at 85% due to some generalization error.

After applying hyperparameter tuning using GridSearchCV, the final training model accuracy decreased to 87%, indicating that the model generalization ability has improved since the testing accuracy has improved to 86%. The final model performed better on unseen data. The final model has addressed potential issues of overfitting in the initial model since the accuracy of the final model is more consistent. The model will accurately predict the outcome 86% of the time.

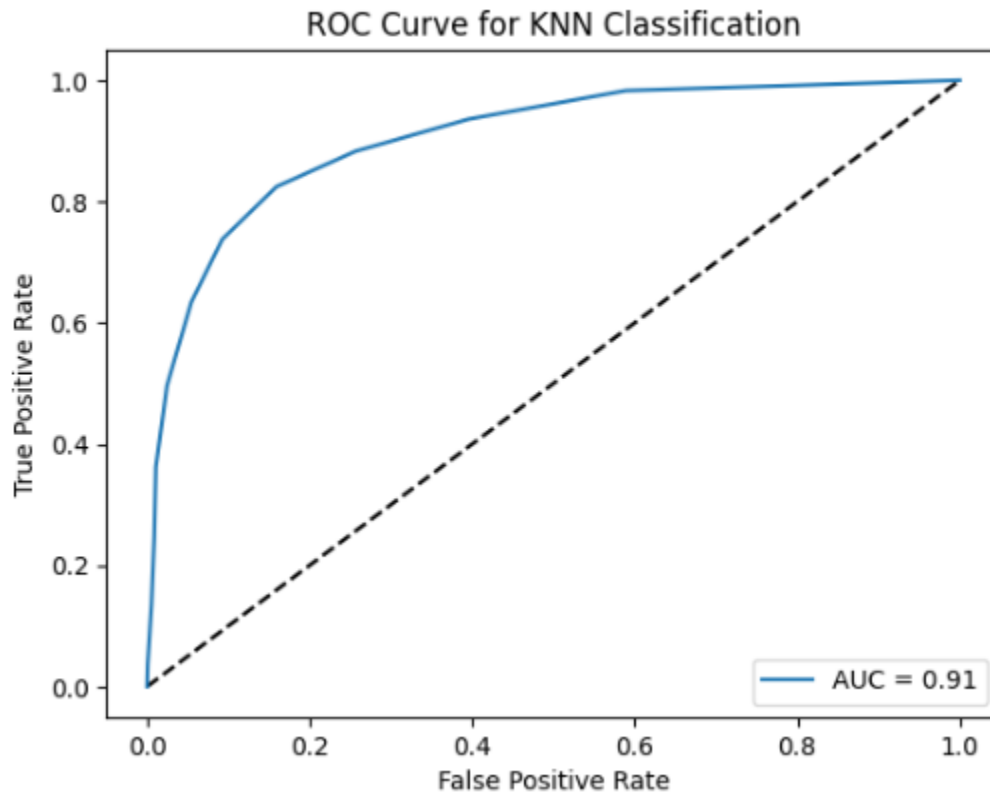
AUC

The Area Under Curve (AUC) is a metric used to evaluate the performance of a classification model. It indicates the ability to differentiate between the positive and negative classes. According to Bhalla, Deepanshu (2014, August), Area under Curve (AUC) or Receiver operating characteristic (ROC) curve is used to evaluate the performance of a binary classification model. It measures the discrimination power of a predictive classification model by checking how well the model is able to distinguish between events and non-events.

Initial ROC Curve:



Final ROC Curve:



The initial AUC was 0.88, indicating that the model ranked a randomly chosen positive instance higher than a randomly chosen negative instance 88% of the time. In contrast, the final AUC was 0.91, indicating that the model can correctly rank a randomly chosen positive instance higher than a randomly chosen negative instance 91% of the time. The higher AUC indicates that the final model is optimized, more reliable, and accurate in predicting scenarios where distinguishing classes is important.

E2: RESULTS AND IMPLICATIONS

The K-Nearest Neighbors (KNN) final model achieved an accuracy of 86% in predicting customer churn for testing (unseen) datasets and 87% for training (seen) data indicating minimal risk of overfitting. Furthermore, when distinguishing between positive (churn) and

negative (non-churn) classes, the model demonstrated a robust Area Under the Curve (AUC) score of 91%, emphasizing its reliability and precision in classification tasks.

The results imply that using this model, we can predict the customers who will churn 86% of the time.

E3:LIMITATION

The analysis was limited due to a lack of understanding of the business data, how the data was collected, and the domain knowledge. This has been observed in the class imbalance (Class 0: 1470, Class 1: 530), likely impacting the model's performance on Class 1.

E4:COURSE OF ACTION

The KNN model performs well with strong accuracy (86%) and ROC-AUC (0.91) but struggles with recall (0.63) and F1-score (0.71) for the minority class due to class imbalance. An understanding of the dataset and further analysis would need to be conducted to improve model performance. With the high accuracy rate, we can predict the customers who are highly likely to churn 86% of the time with minimal risk of overfitting since the accuracy of the training data was 87%.

F: PANOPTO RECORDING

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=7ffdfc2b-5d88-4fa9-a9ec-b26b00f1fdf3>

G: SOURCES FOR THIRD-PARTY CODE

There was no third-party source code used

H: SOURCES

Christopher, Antony (2021, Feb 2) K-Nearest Neighbor

<https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4>

Saxena, Shipra (2024, September 18) What are Categorical Data Encoding Methods | Binary Encoding

<https://www.analyticsvidhya.com/blog/2020/08/types-of-categorical-data-encoding/>

Dilmegani, Cem(2024, October 28) Machine Learning Accuracy: True-False Positive/Negative ['25]

<https://research.aimultiple.com/machine-learning-accuracy/>

Bhalla, Deepanshu (2014, August), A Complete Guide to Area Under Curve (AUC)

<https://www.listendata.com/2014/08/learn-area-under-curve-auc.html>

I: PROFESSIONAL COMMUNICATION